

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра программного обеспечения информационных технологий

ОТЧЁТ
по лабораторной работе №1
по дисциплине «Методы машинного обучения»
Основы программирования на языке Python
Вариант № 12

Выполнил: студент гр. 351002
Емельяненко Е.Д.

Проверил: Лебедев А.В.

Минск 2026

СОДЕРЖАНИЕ

1	Цель выполнения лабораторной работы	4
2	Теория	4
2.1	Ввод, вывод и типы данных	4
2.2	Условные операторы и циклы	4
2.3	Строки	5
2.4	Списки, кортежи и множества	5
2.5	Словари	5
2.6	Функции	5
2.7	Обработка исключений	6
2.8	Объектно-ориентированное программирование	6
3	Формулировка индивидуального задания	6
3.1	Часть 1	6
3.2	Часть 2	7
3.3	Часть 3	7
4	Код решения индивидуального задания	8
4.1	Часть 1, задание 1	8
4.2	Часть 1, задание 2	8
4.3	Часть 1, задание 3	9
4.4	Часть 1, задание 4	9
4.5	Часть 1, задание 5	9
4.6	Часть 1, задание 6	11
4.7	Часть 2, задание 1	11
4.8	Часть 2, задание 2	11
4.9	Часть 2, задание 3	12
4.10	Часть 2, задание 4	12
4.11	Часть 3, задание 1	13
4.12	Часть 3, задание 2	13
4.13	Часть 3, задание 3	14
4.14	Часть 3, задание 4	15
5	Скриншоты выполнения индивидуального задания	16
5.1	Часть 1	16
5.2	Часть 2	19

5.3	Часть 3	21
6	Выводы по лабораторной работе	23

1 ЦЕЛЬ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Изучить основные конструкции языка Python и получить практические навыки их применения: ввод и вывод данных, условные операторы и циклы, работа со строками, списками, кортежами, множествами и словарями, определение и вызов функций, обработка исключений, а также объектно-ориентированное программирование (классы, наследование, переопределение методов, статические методы и методы класса).

2 ТЕОРИЯ

2.1 Ввод, вывод и типы данных

Функция `input()` считывает строку с клавиатуры, а `print()` выводит данные на экран. Считанную строку приводят к нужному типу функциями `int()` и `float()`. Основные арифметические операции: `//` — целочисленное деление, `%` — остаток от деления, `**` — возведение в степень. Форматированный вывод выполняется f-строками: спецификатор `{x:02d}` дополняет целое число нулями до двух знаков, `{x:.2f}` выводит вещественное число с двумя знаками после запятой.

Перевод числа секунд s в формат «дни:часы:минуты:секунды» основан на целочисленном делении и остатке:

$$d = \left\lfloor \frac{s}{86400} \right\rfloor, \quad h = \left\lfloor \frac{s \bmod 86400}{3600} \right\rfloor, \quad m = \left\lfloor \frac{s \bmod 3600}{60} \right\rfloor, \quad c = s \bmod 60.$$

2.2 Условные операторы и циклы

Конструкция `if / elif / else` выбирает ветвь выполнения по логическому условию. Цикл `while` повторяется, пока условие истинно; цикл `for` перебирает элементы последовательности, в том числе получаемой функцией `range()`. Оператор `break` прерывает цикл, `continue` переходит к следующей итерации. Встроенные функции `any()` и `all()` проверяют, выполняется ли условие соответственно хотя бы для одного и для всех элементов последовательности.

2.3 Строки

Строка — неизменяемая последовательность символов. Метод `split()` разбивает строку на слова, `strip()` удаляет пробелы по краям, `capitalize()`, `lower()` и `upper()` изменяют регистр, `count()` считает вхождения символа, `isdigit()` и `isalnum()` проверяют состав строки. Срез `s[::-1]` возвращает строку в обратном порядке. Функция `ord()` возвращает код символа в таблице Unicode. Функция `max(seq, key=len)` находит самый длинный элемент последовательности, а `' '.join(...)` собирает строку из элементов последовательности.

2.4 Списки, кортежи и множества

Список (`list`) — изменяемая упорядоченная последовательность; используются методы `append()` и `reverse()`, срезы, встроенная функция `sum()`. Генератор списка (`[f(x) for x in seq if cond]`) создаёт новый список по правилу. Кортеж (`tuple`) — неизменяемая последовательность. Множество (`set`) — неупорядоченная коллекция уникальных элементов; множества можно сравнивать между собой. Матрица представляется списком списков.

2.5 Словари

Словарь (`dict`) хранит пары «ключ – значение». Метод `items()` возвращает пары для перебора, `get(key, default)` возвращает значение по ключу либо значение по умолчанию. Функция `sorted(d, key=d.get, reverse=True)` сортирует ключи по убыванию значений, а `min(d, key=d.get)` возвращает ключ с минимальным значением.

2.6 Функции

Функция определяется ключевым словом `def`, принимает параметры и возвращает результат оператором `return`. Тип переданного аргумента можно определить выражением `type(x) is list`; это позволяет обрабатывать по-разному списки, словари, числа и строки в одной функции. Срез `x[:] = ...` изменяет содержимое списка на месте, не создавая новый объект.

2.7 Обработка исключений

Исключение — ошибка времени выполнения (`ValueError`, `ZeroDivisionError`). Конструкция `try / except / else / finally` позволяет её обработать: в блоке `try` находится код, который может вызвать ошибку; `except` перехватывает исключение указанного типа; `else` выполняется, если ошибки не было; `finally` выполняется всегда.

2.8 Объектно-ориентированное программирование

Класс описывает набор данных (атрибутов) и операций над ними (методов). Метод `__init__()` — конструктор, вызываемый при создании объекта; первый параметр `self` ссылается на сам объект. Атрибуты экземпляра принадлежат объекту, атрибуты класса общие для всех объектов. Наследование позволяет создать дочерний класс, получающий свойства родительского; функция `super()` обращается к методам родителя, а переопределение метода заменяет его поведение в дочернем классе. Атрибут, имя которого начинается с двух подчёркиваний (`__letters_num`), является приватным. Статический метод (`@staticmethod`) не получает ни объект, ни класс и работает как обычная функция внутри класса; метод класса (`@classmethod`) получает класс первым параметром `cls` и часто используется как альтернативный конструктор. Метод `__str__()` задаёт строковое представление объекта.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

3.1 Часть 1

1. Сделайте так, чтобы число секунд отображалось в виде дни:часы:минуты:секунды. Ввод закончить, когда пользователь ввёл 0.
2. На вход программе подаётся строка текста. Напишите программу, которая переводит каждый её символ в соответствующий ему код из таблицы символов Unicode. Вывести на экран самое длинное слово.
3. На вход программе подаётся натуральное число n , а затем n целых чисел. Напишите программу, которая создаёт из указанных чисел список их кубов. Найти сумму и произведение элементов списка. Вывести список в

обратном порядке.

4. Найдите три ключа с самыми высокими значениями в словаре `my_dict = {'a':500, 'b':5874, 'c': 560, 'd':400, 'e':5874, 'f': 20}`.
5. Реализуйте программу «Магазин игрушек», которая будет включать в себя шесть пунктов меню. Есть словарь, где ключ — название игрушки, значение — список, содержащий состав игрушки, цену и количество (шт.) в магазине. Пункты меню: 1) просмотр описания (название — описание); 2) просмотр цены (название — цена); 3) просмотр количества (название — количество); 4) вся информация; 5) покупка: с клавиатуры вводятся название игрушки и количество, `n` — выход; посчитать цену выбранных товаров и сколько товаров осталось в изначальном списке; 6) до свидания.
6. Дан кортеж целых чисел. Найти сумму чётных элементов.

3.2 Часть 2

1. Написать функцию для определения анаграмм. Два слова являются анаграммами, если состоят из одинаковых букв. Например, анаграммами являются слова «binary» и «brainy», или «раздвоение» и «дозревание».
2. Написать функцию, которая принимает один аргумент. Если в функцию передаётся список, найти сумму всех элементов после последнего положительного и удалить все нулевые элементы из списка. Если словарь — вывести элемент с минимальным значением. Число — вывести в обратном порядке. Строка — определить количество слов. Сделать проверку со всеми этими случаями.
3. Проверить, все ли строки матрицы содержат хотя бы один положительный элемент. Если да, то изменить знаки всех элементов матрицы на обратные.
4. Написать программу, демонстрирующую работу `try / except / finally`.

3.3 Часть 3

1. Создать класс `String` (строка). Разработать в классе методы для работы со строками (не менее 5).

2. Создать класс Alphabet: метод `__init__()` определяет два динамических свойства — `lang` (язык) и `letters` (список букв), значения берутся из входных параметров; метод `print()` выводит буквы алфавита; метод `letters_num()` возвращает количество букв. Создать класс EngAlphabet путём наследования от Alphabet: его метод `__init__()` вызывает родительский `__init__()`, передавая обозначение языка и строку из всех букв алфавита; добавить приватное статическое свойство `__letters_num`; создать метод `is_en_letter()` определяющий, относится ли буква к английскому алфавиту; переопределить метод `letters_num()` так, чтобы он возвращал значение `__letters_num`; создать статический метод `example()`, возвращающий пример текста на английском языке.
3. Реализовать класс Stationery (канцелярская принадлежность): определить в нём атрибут `title` (название) и метод `draw` (отрисовка), выводящий сообщение «Запуск отрисовки»; создать три дочерних класса Pen (ручка), Pencil (карандаш), Handle (маркер); в каждом классе переопределить метод `draw` так, чтобы он выводил уникальное сообщение; создать экземпляры классов и проверить, что выводит метод для каждого экземпляра.
4. Придумать класс самостоятельно, реализовать в нём методы экземпляра класса, статические методы и методы класса.

4 КОД РЕШЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Часть 1, задание 1

```
sec = int(input("Введите количество секунд:"))
while (sec != 0):
    days = sec // (24 * 3600)
    rem = sec % (24 * 3600)
    hours = rem // 3600
    rem = sec % 3600
    minutes = rem // 60
    seconds = sec % 60
    print(f"{days}:{hours:02d}:{minutes:02d}:{seconds:02d}")
    sec = int(input("Введите количество секунд:"))
```

4.2 Часть 1, задание 2

```
text = input().split()
```

```
longest_word = max(text, key=len)
codes = ' '.join(str(ord(char)) for char in longest_word)
print(codes)
```

4.3 Часть 1, задание 3

```
n = int(input("Введите число n: "))

cubes = []
total_sum = 0
product = 1

for i in range(n):
    number = int(input(f"Введите число №{i + 1}: "))
    cube = number ** 3

    cubes.append(cube)
    total_sum += cube
    product *= cube

cubes.reverse()

print(f"Сумма: {total_sum}")
print(f"Произведение: {product}")
print(f"Список в обратном порядке: {cubes}")
```

4.4 Часть 1, задание 4

```
my_dict = {'a': 500, 'b': 5874, 'c': 560, 'd': 400, 'e': 5874, 'f': 20}

result = sorted(my_dict, key=my_dict.get, reverse=True)[:3]
print(result)
```

4.5 Часть 1, задание 5

```
toys = {
    'Мишка': ['плюш, синтепон', 25.5, 10],
    'Машинка': ['пластик, металл', 12.0, 15],
    'Кукла': ['пластик, ткань', 30.0, 7],
    'Конструктор': ['пластик', 45.9, 5],
}

while True:
    print('\n=== Магазин игрушек ===')
    print('1. Просмотр описания')
    print('2. Просмотр цены')
    print('3. Просмотр количества')
    print('4. Вся информация')
    print('5. Покупка')
    print('6. До свидания')
    choice = input('Выберите пункт (1-6): ').strip()
```

```

if choice == '1':
    for name, data in toys.items():
        print(f'{name} - {data[0]}')

elif choice == '2':
    for name, data in toys.items():
        print(f'{name} - {data[1]} руб.')

elif choice == '3':
    for name, data in toys.items():
        print(f'{name} - {data[2]} шт.')

elif choice == '4':
    for name, data in toys.items():
        print(f'{name}: состав - {data[0]}; цена - {data[1]} руб.; количество - {data[2]}  
↪ шт.')

elif choice == '5':
    total = 0
    bought = {}
    while True:
        name = input('Название игрушки (n - завершить покупку): ').strip().capitalize()
        if name.lower() == 'n':
            break
        if name not in toys:
            print('Такой игрушки нет в магазине.')
            continue

        qty_text = input('Количество: ').strip()
        if not qty_text.isdigit() or int(qty_text) <= 0:
            print('Введите целое положительное число.')
            continue
        qty = int(qty_text)

        if qty > toys[name][2]:
            print(f'В наличии только {toys[name][2]} шт.')
            continue

        toys[name][2] -= qty
        total += qty * toys[name][1]
        bought[name] = bought.get(name, 0) + qty
        print(f'Добавлено: {name} x {qty}')

    print('\n--- Чек ---')
    if bought:
        for name, qty in bought.items():
            print(f'{name}: {qty} шт. x {toys[name][1]} = {qty * toys[name][1]:.2f} руб.')
        print(f'Итого к оплате: {total:.2f} руб.')
    else:
        print('Вы ничего не купили.')

    print('\nОстаток в магазине:')
    for name, data in toys.items():
        print(f'{name} - {data[2]} шт.')

elif choice == '6':
    print('До свидания!')
    break

else:
    print('Неверный пункт меню, попробуйте снова.')

```

4.6 Часть 1, задание 6

```
numbers = (3, 8, 12, 5, 7, 4, 6, 0)
total = sum(num for num in numbers if num % 2 == 0)
print(total)
```

4.7 Часть 2, задание 1

```
def check_anagrams(first_word, second_word):
    first_word_set, second_word_set = set(first_word), set(second_word)
    print(first_word_set == second_word_set)
```

```
check_anagrams("brand", "binary")
check_anagrams("brainy", "binary")
```

4.8 Часть 2, задание 2

```
def check_argument(x):
    if type(x) is list:
        last_pos = -1
        for i in range(len(x)):
            if x[i] > 0:
                last_pos = i

        if last_pos == -1:
            print('Положительных элементов нет')
        else:
            print('Сумма после последнего положительного:', sum(x[last_pos + 1:]))

        x[:] = [y for y in x if y != 0]
        print('Список без нулей:', x)
        return x

    elif type(x) is dict:
        if not x:
            print('Словарь пуст')
            return None
        key = min(x, key=x.get)
        print('Минимальный элемент:', key, ':', x[key])
        return key, x[key]

    elif type(x) is int:
        text = str(x)
        if text.startswith('-'):
            rev_text = '-' + text[:0:-1]
        else:
            rev_text = text[::-1]
        result = int(rev_text)
        print(f"Развернутое число: {result}")
        return result

    elif type(x) is str:
        count = len(x.split())
        print('Количество слов:', count)
```

```

        return count

print('--- список без положительных ---')
check_argument([0, -1, 3, -5, 0])

print('--- словарь ---')
check_argument({'a': 500, 'b': 20, 'c': 560})

print('--- число ---')
check_argument(12345)
check_argument(-1200)

print('--- строка ---')
check_argument('Привет, как дела у тебя?')

```

4.9 Часть 2, задание 3

```

def check_matrix(x):
    if all(any(el > 0 for el in row) for row in x):
        x = [[-el for el in row] for row in x]
        print('Знаки изменены:')
    else:
        print('Есть строка без положительных элементов, матрица не изменена:')

    for row in x:
        print(row)

    return x

matrix = [
    [1, -2, 3],
    [-4, 5, -6],
    [-7, -8, 9],
]

check_matrix(matrix)

```

4.10 Часть 2, задание 4

```

def divide_numbers():
    try:
        a = int(input('Введите делимое: '))
        b = int(input('Введите делитель: '))
        result = a / b
    except ValueError:
        print('Ошибка: нужно вводить целые числа.')
    except ZeroDivisionError:
        print('Ошибка: на ноль делить нельзя.')
    else:
        print(f'Результат: {result}')
    finally:
        print('Блок finally выполнен всегда.\n')

```

```
divide_numbers() # 10 и 2 -> Результат: 5.0
divide_numbers() # 10 и 0 -> ZeroDivisionError
divide_numbers() # abc -> ValueError
```

4.11 Часть 3, задание 1

```
class String:

    def __init__(self, text=''):
        self.text = text

    def word_count(self):
        return len(self.text.split())

    def reverse(self):
        return self.text[::-1]

    def is_palindrome(self):
        s = ''.join(ch.lower() for ch in self.text if ch.isalnum())
        return s == s[::-1]

    def count_char(self, char):
        return self.text.count(char)

    def vowels_count(self):
        return sum(1 for ch in self.text.lower() if ch in 'aeiouуаеёиоуэя')

    def longest_word(self):
        words = self.text.split()
        return max(words, key=len) if words else ''

    def __str__(self):
        return self.text

print('=== Задание 1 ===')
s = String('А роза упала на лапу Азора')
print('Слов:', s.word_count())
print('Наоборот:', s.reverse())
print('Палиндром:', s.is_palindrome())
print("Символов 'а':", s.count_char('a'))
print('Гласных:', s.vowels_count())
print('Самое длинное слово:', s.longest_word())
```

4.12 Часть 3, задание 2

```
class Alphabet:

    def __init__(self, lang, letters):
        self.lang = lang
        self.letters = list(letters)

    def print(self):
        print(' '.join(self.letters))
```

```

def letters_num(self):
    return len(self.letters)

class EngAlphabet(Alphabet):
    __letters_num = 26 # приватное статическое свойство

    def __init__(self):
        super().__init__('En', "ABCDEFGHIJKLMNOPQRSTUVWXYZ")

    def is_en_letter(self, letter):
        return len(letter) == 1 and letter.upper() in self.letters

    def letters_num(self):
        return EngAlphabet.__letters_num

    @staticmethod
    def example():
        return 'The quick brown fox jumps over the lazy dog.'

print('\n=== Задание 2 ===')
eng = EngAlphabet()
eng.print()
print('Букв:', eng.letters_num())
print("is_en_letter('F'):", eng.is_en_letter('F'))
print("is_en_letter('Ж'):", eng.is_en_letter('Ж'))
print('Пример:', EngAlphabet.example())

```

4.13 Часть 3, задание 3

```

class Stationery:
    def __init__(self, title):
        self.title = title

    def draw(self):
        print('Запуск отрисовки')

class Pen(Stationery):
    def draw(self):
        print(f'{self.title}: ручка рисует тонкую ровную линию')

class Pencil(Stationery):
    def draw(self):
        print(f'{self.title}: карандаш рисует мягкий штрих')

class Handle(Stationery):
    def draw(self):
        print(f'{self.title}: маркер рисует широкую яркую линию')

print('\n=== Задание 3 ===')
for item in (Stationery('Общая'), Pen('Pen'), Pencil('Pencil'), Handle('Handle')):
    item.draw()

```

4.14 Часть 3, задание 4

```
class BankAccount:
    bank_name = 'SberBank'
    accounts_count = 0

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        BankAccount.accounts_count += 1

    # методы экземпляра
    def deposit(self, amount):
        if not BankAccount.is_valid_amount(amount):
            print('Некорректная сумма')
            return
        self.balance += amount

    def withdraw(self, amount):
        if not BankAccount.is_valid_amount(amount) or amount > self.balance:
            print('Некорректная сумма или недостаточно средств')
            return
        self.balance -= amount

    # статический метод
    @staticmethod
    def is_valid_amount(amount):
        return isinstance(amount, (int, float)) and amount > 0

    # методы класса
    @classmethod
    def from_string(cls, data):
        owner, balance = data.split(';')
        return cls(owner.strip(), float(balance))

    @classmethod
    def total_accounts(cls):
        return cls.accounts_count

    def __str__(self):
        return f'{self.bank_name}: {self.owner}, баланс {self.balance:.2f}'

print('\n=== Задание 4 ===')
acc = BankAccount('Иван', 100)
acc.deposit(50)
acc.withdraw(30)
acc.withdraw(1000)
print(acc)

acc2 = BankAccount.from_string('Пётр; 500')
print(acc2)
print('Всего счетов:', BankAccount.total_accounts())
```

5 СКРИНШОТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

5.1 Часть 1

Введите количество секунд: 360000
4:04:00:00
Введите количество секунд: 350000
4:01:13:20
Введите количество секунд: 300000
3:11:20:00
Введите количество секунд: 369875
4:06:44:35
Введите количество секунд: 0

Рис. 1 – Часть 1, задание 1: перевод секунд в формат
дни:часы:минуты:секунды

Hello world!!!!
119 111 114 108 100 33 33 33 33

Рис. 2 – Часть 1, задание 2: коды символов самого длинного слова

Введите число n: 4
Введите число №1: 5
Введите число №2: 6
Введите число №3: 6
Введите число №4: 6
Сумма: 773
Произведение: 1259712000
Список в обратном порядке: [216, 216, 216, 125]

Рис. 3 – Часть 1, задание 3: кубы чисел, сумма, произведение и обратный порядок

['b', 'e', 'c']

Рис. 4 – Часть 1, задание 4: три ключа с наибольшими значениями

```

=== Магазин игрушек ===
1. Просмотр описания
2. Просмотр цены
3. Просмотр количества
4. Вся информация
5. Покупка
6. До свидания
Выберите пункт (1-6): 5
Название игрушки (n - завершить покупку): кукла
Количество: 3
Добавлено: Кукла x 3
Название игрушки (n - завершить покупку): n

--- Чек ---
Кукла: 3 шт. x 30.0 = 90.00 руб.
Итого к оплате: 90.00 руб.

Остаток в магазине:
Мишка - 10 шт.
Машинка - 15 шт.
Кукла - 4 шт.
Конструктор - 5 шт.

=== Магазин игрушек ===
1. Просмотр описания
2. Просмотр цены
3. Просмотр количества
4. Вся информация
5. Покупка
6. До свидания
Выберите пункт (1-6): 4
Мишка: состав - плюш, синтепон; цена - 25.5 руб.; количество - 10 шт.
Машинка: состав - пластик, металл; цена - 12.0 руб.; количество - 15 шт.
Кукла: состав - пластик, ткань; цена - 30.0 руб.; количество - 4 шт.
Конструктор: состав - пластик; цена - 45.9 руб.; количество - 5 шт.

=== Магазин игрушек ===
1. Просмотр описания
2. Просмотр цены
3. Просмотр количества
4. Вся информация
5. Покупка
6. До свидания
Выберите пункт (1-6): 6
До свидания!

```

Рис. 5 – Часть 1, задание 5: программа «Магазин игрушек»

▼ Задание №6

Дан кортеж целых чисел. Найти сумму чётных элементов.

```

[8]: numbers = (3, 8, 12, 5, 7, 4, 6, 0)
total = sum(num for num in numbers if num % 2 == 0)
print(total)

```

30

Рис. 6 – Часть 1, задание 6: сумма чётных элементов кортежа

5.2 Часть 2

```
[14]: def check_anagrams(first_word, second_word):
      first_word_set, second_word_set = set(first_word), set(second_word)
      print(first_word_set == second_word_set)

      check_anagrams("brand", "binary")
      check_anagrams("rainy", "binary")

False
True
```

Рис. 7 – Часть 2, задание 1: проверка анаграмм

```
print('--- список ---')
check_argument([0, -1, 3, -5, 0])

print('--- словарь ---')
check_argument({'a': 500, 'b': 20, 'c': 560})

print('--- число ---')
check_argument(12345)
check_argument(-1200)

print('--- строка ---')
check_argument('Привет, как дела у тебя?')
```

--- список ---
Сумма после последнего положительного: -5
Список без нулей: [-1, 3, -5]
--- словарь ---
Минимальный элемент: b : 20
--- число ---
Развернутое число: 54321
Развернутое число: -21
--- строка ---
Количество слов: 5

Рис. 8 – Часть 2, задание 2: функция с разной обработкой аргументов

```
matrix = [
    [1, -2, 3],
    [-4, 5, -6],
    [-7, -8, 9],
]

check_matrix(matrix)
```

Знаки изменены:

```
[-1, 2, -3]
[4, -5, 6]
[7, 8, -9]
```

Рис. 9 – Часть 2, задание 3: проверка строк матрицы

```
: def divide_numbers():
    try:
        a = int(input('Введите делимое: '))
        b = int(input('Введите делитель: '))
        result = a / b
    except ValueError:
        print('Ошибка: нужно вводить целые числа.')
    except ZeroDivisionError:
        print('Ошибка: на ноль делить нельзя.')
    else:
        print(f'Результат: {result}')
    finally:
        print('Блок finally выполнен всегда.\n')

divide_numbers()
divide_numbers()
divide_numbers()

Введите делимое: 10
Введите делитель: 5
Результат: 2.0
Блок finally выполнен всегда.

Введите делимое: 10
Введите делитель: 0
Ошибка: на ноль делить нельзя.
Блок finally выполнен всегда.

Введите делимое: abc
Ошибка: нужно вводить целые числа.
Блок finally выполнен всегда.
```

Рис. 10 – Часть 2, задание 4: блоки try / except / finally

5.3 Часть 3

```
print('=== Задание 1 ===')
s = String('А роза упала на лапу Азора')
print('Слов:', s.word_count())
print('Наоборот:', s.reverse())
print('Палиндром:', s.is_palindrome())
print("Символов 'а':", s.count_char('a'))
print('Гласных:', s.vowels_count())
print('Самое длинное слово:', s.longest_word())
```

```
=== Задание 1 ===
Слов: 6
Наоборот: арозА упал ан алапу азор А
Палиндром: True
Символов 'а': 6
Гласных: 12
Самое длинное слово: упала
```

Рис. 11 – Часть 3, задание 1: класс String

```
print('\n=== Задание 2 ===')
eng = EngAlphabet()
eng.print()
print('Букв:', eng.letters_num())
print("is_en_letter('F'):", eng.is_en_letter('F'))
print("is_en_letter('Ж'):", eng.is_en_letter('Ж'))
print('Пример:', EngAlphabet.example())
```

```
=== Задание 2 ===
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Букв: 26
is_en_letter('F'): True
is_en_letter('Ж'): False
Пример: The quick brown fox jumps over the lazy dog.
```

Рис. 12 – Часть 3, задание 2: классы Alphabet и EngAlphabet

```
print('\n=== Задание 3 ===')
for item in (Stationery('Общая'), Pen('Pen'), Pencil('Pencil'), Handle('Handle')):
    item.draw()
```

```
=== Задание 3 ===
Запуск отрисовки
Pen: ручка рисует тонкую ровную линию
Pencil: карандаш рисует мягкий штрих
Handle: маркер рисует широкую яркую линию
```

Рис. 13 – Часть 3, задание 3: класс Stationery и его наследники

```
print('\n=== Задание 4 ===')
acc = BankAccount('Иван', 100)
acc.deposit(50)
acc.withdraw(30)
acc.withdraw(1000)
print(acc)

acc2 = BankAccount.from_string('Пётр; 500')
print(acc2)
print('Всего счетов:', BankAccount.total_accounts())
```

```
=== Задание 4 ===
Некорректная сумма или недостаточно средств
SberBank: Иван, баланс 120.00
SberBank: Пётр, баланс 500.00
Всего счетов: 2
```

Рис. 14 – Часть 3, задание 4: класс BankAccount

6 ВЫВОДЫ ПО ЛАБОРАТОРНОЙ РАБОТЕ

В ходе лабораторной работы (вариант 12) выполнены задания трёх частей на языке Python.

В первой части реализованы программы с вводом данных и циклами: перевод числа секунд в формат «дни:часы:минуты:секунды» с помощью целочисленного деления и остатка, вывод кодов Unicode для символов самого длинного слова строки, вычисление списка кубов с суммой, произведением и обратным порядком, выбор трёх ключей словаря с наибольшими значениями, меню программы «Магазин игрушек» с покупкой и подсчётом остатка, а также сумма чётных элементов кортежа.

Во второй части написаны функции: проверка анаграмм с помощью множеств, обработка аргумента в зависимости от его типа (список, словарь, число, строка), проверка строк матрицы на наличие положительного элемента с изменением знаков, а также программа, демонстрирующая работу блоков `try / except / else / finally` при делении чисел.

В третьей части реализованы классы: `String` с методами для работы со строками, `Alphabet` и наследующий его `EngAlphabet` (с приватным статическим свойством, переопределением метода и статическим методом), `Stationery` с дочерними классами `Pen`, `Pencil` и `Handle`, переопределяющими метод `draw`, а также собственный класс `BankAccount` с методами экземпляра, статическим методом и методами класса.

Все программы выполнены, результаты работы подтверждены скриншотами. Лабораторная работа позволила закрепить базовые конструкции языка Python, работу со структурами данных, функциями, исключениями и основами объектно-ориентированного программирования.